CO3 AT2 - Experiments – Actor-Critic Methods (A2C, A3C)

A2C – Advantage Actor-Critic

Definition:
A2C (Advantage Actor-Critic) is a reinforcement learning algorithm that combines policy-based learning (Actor) and value-based learning (Critic). The Actor selects actions, while the Critic evaluates how good those actions are using the advantage function.

Key points:

- Actor: Decides which action to take.
- Critic: Evaluates the current state/action.
- Uses the advantage value to improve the policy.
- Performs synchronous updates.
- Provides relatively stable and efficient learning.

Simple example:
In a game, the Actor decides whether to move left or right, while the Critic determines whether that decision was good or bad based on the reward.

Code :

```
import gymnasium as gym
import torch
import torch.nn as nn
import torch.optim as optim
import numpy as np
import matplotlib.pyplot as plt

# Create environment
env = gym.make("CartPole-v1")

# Get state and action sizes
state_size = env.observation_space.shape[0]
action_size = env.action_space.n

# Actor-Critic Neural Network
class ActorCritic(nn.Module):
    def __init__(self, state_size, action_size):
        super().__init__()

        self.shared = nn.Sequential(nn.Linear(state_size, 128),nn.ReLU())

        # Actor: gives probability of actions
        self.actor = nn.Linear(128, action_size)
```

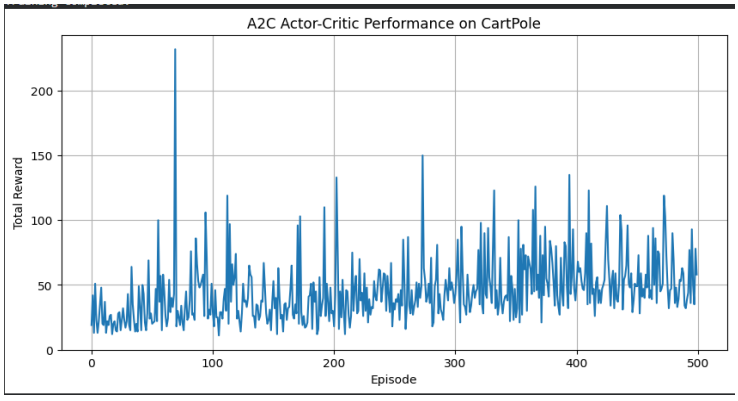
```
        # Critic: estimates value of state
        self.critic = nn.Linear(128, 1)

    def forward(self, state):
        x = self.shared(state)

        action_probs = torch.softmax(self.actor(x), dim=-1)
        state_value = self.critic(x)

        return action_probs, state_value


# Create model
model = ActorCritic(state_size, action_size)

optimizer = optim.Adam(model.parameters(), lr=0.001)

gamma = 0.99
episodes = 500

episode_rewards = []

print("Training A2C Actor-Critic...")

# Training
for episode in range(episodes):

    state, info = env.reset()

    log_probs = []
    values = []
    rewards = []

    done = False

    while not done:

        state_tensor = torch.tensor(state, dtype=torch.float32).unsqueeze(0)

        # Actor-Critic prediction
        action_probs, value = model(state_tensor)

        # Select action
        distribution = torch.distributions.Categorical(action_probs)

        action = distribution.sample()
```

```
    log_prob = distribution.log_prob(action)

    # Take action
    next_state, reward, terminated, truncated, info = env.step(action.item())

    done = terminated or truncated

    # Store information
    log_probs.append(log_prob)
    values.append(value.squeeze())
    rewards.append(reward)

    state = next_state

# Calculate returns
returns = []
G = 0

for reward in reversed(rewards):
    G = reward + gamma * G
    returns.insert(0, G)

returns = torch.tensor(returns,dtype=torch.float32)

values = torch.stack(values)
log_probs = torch.stack(log_probs)

# Advantage
advantages = returns - values.detach()

# Actor loss
actor_loss = -(log_probs * advantages).mean()

# Critic loss
critic_loss = nn.functional.mse_loss(values,return)

# Total loss
loss = actor_loss + 0.5 * critic_loss

# Update model
optimizer.zero_grad()
loss.backward()
optimizer.step()

total_reward = sum(rewards)
episode_rewards.append(total_reward)
```

```
    if (episode + 1) % 50 == 0:
        average_reward = np.mean(episode_rewards[-50:])

        Print("Episode:", episode + 1, "Average Reward:",round(average_reward, 2))

env.close()

print("\nTraining completed!")

# Plot results
plt.figure(figsize=(10, 5))

plt.plot(episode_rewards)

plt.xlabel("Episode")
plt.ylabel("Total Reward")

plt.title("A2C Actor-Critic Performance on CartPole")

plt.grid(True)

plt.show()
```

Output:

```
Training A2C Actor-Critic...
Episode: 50 Average Reward: 26.86
Episode: 100 Average Reward: 43.46
Episode: 150 Average Reward: 37.94
Episode: 200 Average Reward: 37.2
Episode: 250 Average Reward: 42.64
Episode: 300 Average Reward: 45.18
Episode: 350 Average Reward: 49.84
Episode: 400 Average Reward: 59.26
Episode: 450 Average Reward: 56.34
Episode: 500 Average Reward: 56.06

Training completed!
```



A3C – Asynchronous Advantage Actor-Critic

Definition:
A3C (Asynchronous Advantage Actor-Critic) is a reinforcement learning algorithm in which multiple agents/workers learn simultaneously and asynchronously. Each worker interacts with its own environment and updates a shared global Actor-Critic model.

Key points:

- Uses multiple independent workers.
- Each worker explores the environment separately.
- Workers calculate gradients from their experiences.
- Updates are sent to a shared global model asynchronously.
- Different workers provide diverse experiences, improving exploration.

Simple example:
Imagine several students learning the same game strategy independently. Each student tries different actions and reports what they learned to a common teacher. The teacher combines their learning to improve the overall strategy.

Code:

```
import gymnasium as gym
import torch
import torch.nn as nn
import torch.optim as optim
import torch.multiprocessing as mp
import matplotlib.pyplot as plt
import numpy as np

# --------------------------------------------------
# Actor-Critic Network
# --------------------------------------------------

class ActorCritic(nn.Module):

  def __init__(self, state_size, action_size):
    super(ActorCritic, self).__init__()

    self.fc = nn.Linear(state_size, 128)

    # Actor
    self.actor = nn.Linear(128, action_size)

    # Critic
    self.critic = nn.Linear(128, 1)
```

```
    def forward(self, state):

        x = torch.relu(self.fc(state))

        action_probs = torch.softmax(self.actor(x), dim=-1)

        value = self.critic(x)

        return action_probs, value
# ------------------------------------------------
# Worker Function
# ------------------------------------------------

def worker(worker_id, global_model, optimizer, rewards_list):

    env = gym.make("CartPole-v1")

    state_size = env.observation_space.shape[0]
    action_size = env.action_space.n

    local_model = ActorCritic(state_size,action_size)

    gamma = 0.99

    for episode in range(100):

        state, info = env.reset()

        log_probs = []
        values = []
        rewards = []

        done = False

        while not done:

            state_tensor = torch.tensor(state,dtype=torch.float32 ).unsqueeze(0)

            # Use global model for prediction
            action_probs, value = global_model(state_tensor)

            # Select action
            distribution = torch.distributions.Categorical(action_probs)
            action = distribution.sample()

            log_prob = distribution.log_prob(action)
```

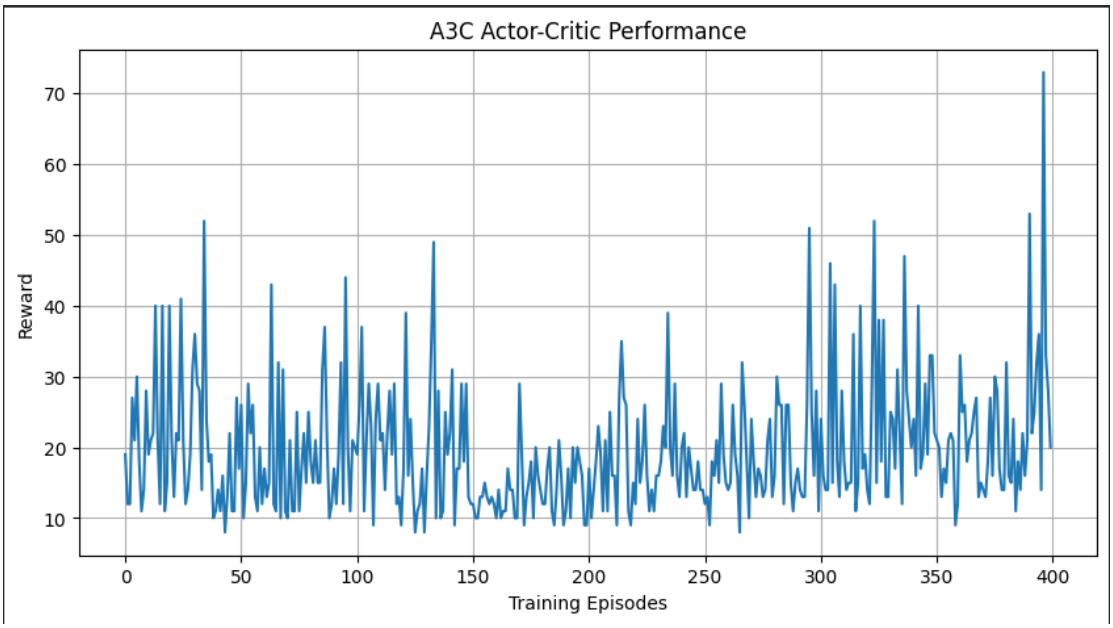
```
    # Perform action
    next_state, reward, terminated, truncated, info = env.step(action.item())
    done = terminated or truncated

    log_probs.append(log_prob)
    values.append(value.squeeze())
    rewards.append(reward)

    state = next_state

# ----------------------------------------
# Calculate discounted returns
# ----------------------------------------

returns = []
G = 0

for reward in reversed(rewards):

    G = reward + gamma * G
    returns.insert(0, G)

returns = torch.tensor(returns,dtype=torch.float32)

values = torch.stack(values)
log_probs = torch.stack(log_probs)

# Advantage
advantages = returns - values.detach()

# Actor loss
actor_loss = -(log_probs * advantages).mean()

# Critic loss
critic_loss = nn.functional.mse_loss(values,returns)

# Total loss
loss = actor_loss + 0.5 * critic_loss

# ----------------------------------------
# Update global model
# ----------------------------------------

optimizer.zero_grad()

loss.backward()
```

```
            optimizer.step()

            total_reward = sum(rewards)

            rewards_list.append((worker_id, total_reward))

            if episode % 20 == 0:

                print("Worker:",worker_id,"Episode:",episode,"Reward:", total_reward)

        env.close()

    # ------------------------------------------------
    # Main Program
    # ------------------------------------------------

    if __name__ == "__main__":

        # Environment information
        env = gym.make("CartPole-v1")

        state_size = env.observation_space.shape[0]
        action_size = env.action_space.n

        env.close()

        # Global Actor-Critic model
        global_model = ActorCritic(state_size, action_size)

        # Shared memory
        global_model.share_memory()

        # Optimizer
        optimizer = optim.Adam(global_model.parameters(),lr=0.001)

        # Store rewards
        rewards_list = mp.Manager().list()

        # Number of workers
        num_workers = 4

        processes = []

        print("Starting A3C training...\n")

        # Create workers
```

```
for worker_id in range(num_workers):

    process = mp.Process(target=worker,args=(worker_id,global_model, optimizer,
          rewards_list))

    process.start()

    processes.append(process)

# Wait for workers
for process in processes:
    process.join()

print("\nA3C Training Completed!")

# ------------------------------------------------
# Plot results
# ------------------------------------------------

rewards = [x[1] for x in rewards_list]

plt.figure(figsize=(10, 5))

plt.plot(rewards)

plt.xlabel("Training Episodes")
plt.ylabel("Reward")

plt.title( "A3C Actor-Critic Performance")

plt.grid()

plt.show()
```

Output:

```
Starting A3C training...

Worker: Worker:1  0Episode: Episode:  0 0Reward:  Reward: 19.012.0

Worker: 2 Episode: 0 Reward: 12.0
Worker: 3 Episode: 0 Reward: 30.0
Worker: 0 Episode: 20 Reward: 11.0
Worker: 1 Episode: 20 Reward: 25.0
Worker: 2 Episode: 20 Reward: 15.0
Worker: 3 Episode: 20 Reward: 31.0
Worker: Worker:2 3 Episode: Worker:Episode:   4040 1  Reward:Reward: Episode: 14.012.0

40 Worker:Reward:  10.0
0 Episode: 40 Reward: 11.0
Worker: 3 Episode: 60 Reward: 11.0
Worker: 2 Episode: Worker:60 1  Episode: 60 Reward:Reward: 14.0
20.0Worker:
 0 Episode: 60 Reward: 14.0
Worker: 3 Episode: 80 Reward: 18.0
Worker: 2 Episode: 80 Reward: 40.0
Worker: 1 Episode: 80 Reward: 29.0
Worker: 0 Episode: 80 Reward: 15.0

A3C Training Completed!
```


A3C Actor-Critic Performance
Reward
Training Episodes
10
20
30
40
50
60
70
0
50
100
150
200
250
300
350
400